

Disassembly sequence planning using a Simplified Teaching–Learning-Based Optimization algorithm[☆]

Kai Xia^a, Liang Gao^{a,*}, Weidong Li^b, Kuo-Ming Chao^b

^a The State Key Laboratory of Digital Manufacturing Equipment and Technology, Huazhong University of Science and Technology, Wuhan 430074, China

^b Faculty of Engineering and Computing, Coventry University, Coventry CV1 5FB, UK

ARTICLE INFO

Article history:

Received 1 February 2013

Received in revised form 15 July 2014

Accepted 18 July 2014

Available online 11 August 2014

Keywords:

Disassembly

Disassembly sequence planning

Meta-heuristics

Teaching–Learning-Based Optimization

Simplified Teaching–Learning-Based

Optimization

ABSTRACT

Disassembly Sequence Planning (DSP) is a challenging NP-hard combinatorial optimization problem. As a new and promising population-based evolutionary algorithm, the Teaching–Learning-Based Optimization (TLBO) algorithm has been successfully applied to various research problems. However, TLBO is not capable or effective in DSP optimization problems with discrete solution spaces and complex disassembly precedence constraints. This paper presents a Simplified Teaching–Learning-Based Optimization (STLBO) algorithm for solving DSP problems effectively. The STLBO algorithm inherits the main idea of the teaching–learning-based evolutionary mechanism from the TLBO algorithm, while the realization method for the evolutionary mechanism and the adaptation methods for the algorithm parameters are different. Three new operators are developed and incorporated in the STLBO algorithm to ensure its applicability to DSP problems with complex disassembly precedence constraints: i.e., a Feasible Solution Generator (FSG) used to generate a feasible disassembly sequence, a Teaching Phase Operator (TPO) and a Learning Phase Operator (LPO) used to learn and evolve the solutions towards better ones by applying the method of precedence preservation crossover operation. Numerical experiments with case studies on waste product disassembly planning have been carried out to demonstrate the effectiveness of the designed operators and the results exhibited that the developed algorithm performs better than other relevant algorithms under a set of public benchmarks.

© 2014 Elsevier Ltd. All rights reserved.

1. Introduction

Mass-customized productions, technology updating and shortening lifespan of products in modern societies have resulted in generation of enormous amount of waste products like Waste Electrical and Electronic Equipment (WEEE). Developing technical solutions for sustainable recovery of waste products becomes a global trend. End-of-life recovery options include part reuse, remanufacturing, material recycling, energy recovery and disposal. As shown in Fig. 1, disassembly, which is a systematic method for separating a product into its constituent components and subassemblies [1], is a critical stage for end-of-life recovery. Finding an optimum or near optimum disassembly sequence is crucial to increasing the efficiency of the disassembly process.

Disassembly Sequence Planning (DSP) determines the order in which components are removed from products aiming at minimizing the disassembly time or cost, while considering the

disassembly direction, disassembly method, and other attributes of components. DSP has been proved as a NP-hard problem [2] and has been becoming an important but still a challenging research topic in recent years. In the previous research, heuristics and meta-heuristics were used to find near optimum or optimum solutions and generate cost-effective and feasible disassembly sequences. Heuristics include rule-based recursive method [3], graph-based heuristic approach [4], etc. Meta-heuristics, which have been widely applied for solving such problems as well, include Genetic Algorithm (GA) [5–11], Particle Swarm Optimization (PSO) [12–17], Ant Colony Optimization (ACO) [18–20], Greedy Randomized Adaptive Search Procedure (GRASP) [21–23], etc. However, the controlling parameters in the above meta-heuristics need to be tuned, such as crossover rate and mutation probability in GA, inertia weight and two acceleration constants in PSO, two weights for path selection and evaporation rate in ACO, and window size in GRASP, etc. This characteristic makes the above meta-heuristics not adaptive and robust enough for various situations.

A new population-based evolutionary algorithm named Teaching–Learning-Based Optimization (TLBO) algorithm, which

[☆] Handled by C.-H. Chen.

* Corresponding author. Tel.: +86 2787559419; fax: +86 2787543074.

E-mail address: gaoliang@mail.hust.edu.cn (L. Gao).

Nomenclature

ACO	Ant Colony Optimization	ROB	Rate of Best
ART	Average Running Time	SSO	Simplified Swarm Optimization
BOM	Bill of Materials	STD	Standard Deviation
DSP	Disassembly Sequence Planning	STLBO	Simplified Teaching–Learning–Based Optimization
FSG	Feasible Solution Generator	TLBO	Teaching–Learning–Based Optimization
GA	Genetic Algorithm	TPO	Teaching Phase Operator
GRASP	Greedy Randomized Adaptive Search Procedure		
LPO	Learning Phase Operator		
PSO	Particle Swarm Optimization		

was originally introduced by Rao, et al. in 2012 [24,25], has been successfully applied to continuous non-linear large scale problems [26,27] including mechanical design optimization [25], parameter optimization of machining processes [28–30], high dimensional real parameter optimization [31], economic emission load dispatch [32], etc. Unlike the above optimization techniques, the TLBO algorithm does not require any algorithm parameters (except population size and iteration times) to be tuned and outperforms some of the well-known meta-heuristics regarding constrained benchmark functions, constrained mechanical design, and continuous non-linear numerical optimization problems [24].

However, the TLBO algorithm is not suitable for solving the DSP problems directly as it was designed for continuous optimization problems while the DSP problems are discrete combinatorial optimization problems with complex disassembly precedence constraints. In continuous problems, a solution is a vector of design variables, each of which belongs to a continuous range. Sequencing the components in a product for disassembly planning is a typical DSP problem. In this problem, a solution can be represented as a permutation of integers, which are the serial numbers of components in a product. For this DSP problem, the solution space is not continuous and TLBO is unable to be applied directly. Meanwhile, a directly discretized TLBO algorithm could not be a good choice for the above DSP problem either. In a DSP problem, the search space for an optimal solution is growing exponentially according to the number of components in the product, while the disassembly constraints could be complex. These characteristics cause that there are few feasible solutions in the population by using a random solution generation method embedded in TLBO. Furthermore, disassembly precedence constraints cannot be preserved simply using an arithmetic operation method during the evolutions towards optimization, which leads to few feasible solutions in the offspring. Hence, the directly discretized TLBO algorithm can hardly converge and solve the DSP problem effectively.

In order to solve the DSP problems more efficiently, this paper proposes a new optimization algorithm named Simplified Teaching–Learning–Based Optimization (STLBO) algorithm. The STLBO algorithm inherits the main idea of the teaching–learning-based evolutionary mechanism so as to take the merits of the TLBO algorithm. Three new operators, including a Feasible

Solution Generator (FSG), a Teaching Phase Operator (TPO) and a Learning Phase Operator (LPO), have been designed and incorporated into the algorithm so as to make the algorithm applicable for DSP problems with complex constraints. In the meantime, the complex and multidimensional matrix computation used to modify solutions in the TLBO algorithm is simplified to a precedence preservation crossover operation in the STLBO algorithm. The detecting of feasibilities of new generated solutions would be also avoided. With the designed operators, STLBO can converge faster in the optimization or search process with higher accuracy so as to eventually improve the disassembly efficiency as well as reducing disassembly cost.

The rest of the paper is organized as follows: In Section 2, the proposed STLBO algorithm for DSP problems is presented in detail. Section 3 demonstrates the performance of STLBO algorithm through numerical experiments and benchmark tests with case studies of waste product disassembly planning. Finally, conclusions are drawn in Section 4.

2. STLBO algorithm for DSP problems

This section presents the STLBO algorithm developed for DSP problems in detail. Firstly, the TLBO algorithm is introduced briefly. And then, the framework of the STLBO algorithm is described. Subsequently, three key operators implemented in STLBO algorithm are presented in detail with illustrative examples of disassembly of waste products. Lastly, the comparison analysis of STLBO algorithm with other optimization algorithms is given and the implementation steps of STLBO algorithm for the DSP problems are outlined.

2.1. Brief introduction to the TLBO algorithm

The TLBO algorithm is a teaching–learning process inspired method simulating two fundamental modes of learning: through teacher (teaching phase) and interacting with the other learners (learning phase), where a group of learner (i.e. class) is considered as a population and the different learning subjects offered to the learners represents the different decision variables of the optimization problem. The grade of a learning subject offered to a learner represents the value of a decision variable in a solution. The result of a learner calculated from the grades of all the offered learning subjects represents the fitness function value of a solution. The learner with the best result in the entire population is considered as the teacher. The TLBO algorithm is explained below with the teaching phase and learning phase.

In the teaching phase, a good teacher conveys knowledge among the learners and tries to improve the mean results of the learners. Suppose there are m learning subjects (decision variables) offered to n learners in a class. At any sequential teaching–learning cycle t , the grades of learner i ($i = 1, 2, \dots, n$) is denoted as $X_i^t = (x_{i,1}^t, x_{i,2}^t, \dots, x_{i,m}^t)$, where $x_{i,j}^t$ denotes the grade of learner i in

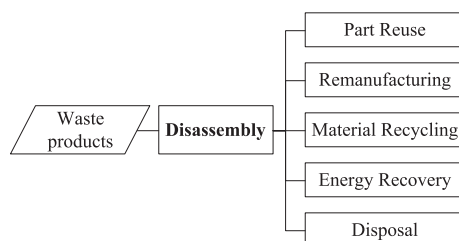


Fig. 1. End-of-life recovery options for waste products.

subject j ($j = 1, 2, \dots, m$) in cycle t . The mean grade of all the learners in subject j is denoted as M_j^t . The result of learner i considering all the subjects is denoted as $f(X_i^t)$. The grades of the teacher equals to those of the best learner X_{best}^t . The difference between the grade of the teacher and mean grade of the learners in each subject $D^t = (d_1^t, d_2^t, \dots, d_m^t)$ is calculated by the following equation:

$$d_j^t = r_t \cdot (x_{best,j}^t - \text{round}(1 + r_t) \cdot M_j^t) \quad (1)$$

where r_t is a random number in the range $[0, 1]$ and $\text{round}(1 + r_t)$ can be either 1 or 2 decided by r_t . Based on D^t , X_i^t is modified according to the following equation:

$$X_i^{new,t} = X_i^t + D^t \quad (2)$$

If $X_i^{new,t}$ gives a better result (fitness value) than X_i^t , update X_i^t with $X_i^{new,t}$.

In the learning phase, a learner p may learn something new from another randomly selected learner q ($q \neq p$) according to the following equation:

$$X_p^{new,t} = \begin{cases} X_p^t + r_t(X_p^t - X_q^t), & \text{if } (f(X_p^t) < f(X_q^t)) \\ X_p^t + r_t(X_q^t - X_p^t), & \text{otherwise} \end{cases} \quad (3)$$

If $X_p^{new,t}$ gives a better result (fitness value) than X_p^t , update X_p^t with $X_p^{new,t}$.

As analyzed in the Introduction section, neither directly implementation nor directly discretization of the TLBO algorithm could solve the DSP problems effectively. The new proposed STLBO algorithm is presented in detail in the following subsections.

2.2. Framework of the STLBO algorithm

The STLBO algorithm is based on the framework of a discrete and population-based optimization method, and inherits the main idea of the teaching–learning-based evolutionary mechanism from the TLBO algorithm. The STLBO algorithm includes two phase: teaching phase and learning phase, and three key parts: FSG, TPO and LPO. The pseudo-code of the STLBO algorithm is illustrated in Fig. 2.

```

STLBO
1 //Initialization:
2 For  $i = 1$  to class size do
3   Generate learner  $X_i$  using FSG;
4   Calculate the fitness function value of each learner  $f(X_i)$ ;
5 End for
6 //Loop:
7 While (stop criterion is not satisfied & maximum iterations are not exceeded) do
8   //Teaching phase:
9   Identify the teacher;
10  Calculate the mean of the fitness function values of learners;
11  Calculate the teaching factor;
12  For  $i = 1$  to class size do
13    Update learner  $X_i$  using TPO;
14  End for
15 //Learning phase:
16 For  $i = 1$  to class size do
17   Randomly select another learner  $X_j$ ;
18   Generate a new learner  $X_r$  using FSG;
19   Calculate  $f(X_r)$ ;
20   Calculate the self-learning factor and factor of learning from another;
21   Update learner  $X_i$  using LPO;
22 End for
23 End while

```

Fig. 2. The pseudo-code of the STLBO algorithm.

In the teaching phase, the teacher is identified firstly, and then the mean of the fitness function values (mean result) of the learners is calculated. According to the mean result and the result of the teacher, the teaching factor is calculated, which is a self-adaptive parameter determining the probability that a learner learns from the teacher. Each learner will be modified using TPO according to the teaching factor.

In the learning phase, another learner is randomly selected for each learner and a new learner is generated using FSG with its result calculated. According to the results of these three learners, self-learning factor and the factor of learning from another are calculated, which are self-adaptive parameters respectively determine the probability that a learner learns from himself, and the probability that a learner learns from another one.

The detailed explanations of FSG, TPO and LPO are presented in the following subsections.

2.3. Feasible solution generator

A feasible solution for a DSP problem is a disassembly sequence that satisfies the given disassembly precedence constraints. The implementation model of FSG depends on the way how disassembly precedence constraints are represented. The general steps for implementing the FSG are as follows:

- Step 1: Set the vector of components available for disassembly C_A to $\emptyset$;
- Step 2: Check the current disassembly state and disassembly precedence constraints to find new components available for disassembly, and push them back to C_A ;
- Step 3: Choose one component from C_A randomly, push it back to Solution, erase it from C_A and suppose that it is disassembled from the product;
- Step 4: If all the components are disassembled from the product, the Solution is randomly generated, otherwise, go to Step 2.

This paper studied two types of DSP problems with different ways of representing the disassembly precedence constraints. The aforementioned 4 steps are general ones for implementing FSG; however, disassembly precedence constraints are represented in different ways, so the methods for finding the available components for disassembly by checking disassembly precedence constraints vary.

Problem 1. All the components and joints in the waste product are uniformly considered as components, in other words, the components and joints in the waste product are not discriminated. For instance, in the example illustrated in Fig. 3, joints j_1 and j_2 both are considered as components and represented as c_4 and c_5

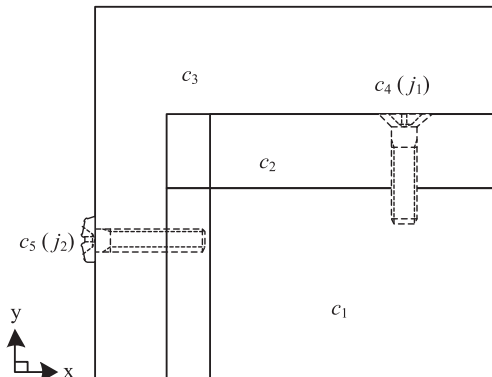


Fig. 3. An example of 2D product model.

respectively. The disassembly precedence constraints are represented as rules, which are described as pairs of precedent set Pre_i and following set Fol_i . In each rule i , the components in Pre_i have disassembly precedence over the components in Fol_i . The disassembly precedence rules for the example in Fig. 3 are as follows:

$Rule_1: Pre_1 = \emptyset, Fol_1 = \{c_5\};$

$Rule_2: Pre_2 = \{c_5\}, Fol_2 = \{c_3\};$

$Rule_3: Pre_3 = \{c_3\}, Fol_3 = \{c_4\};$

$Rule_4: Pre_4 = \{c_4\}, Fol_4 = \{c_1, c_2\}.$

The pseudo-code of FSG implemented for Problem 1 is outlined in Fig. 4.

Problem 2. The components and joints in the waste product are considered discriminatorily. In the disassembly process, the components are to be removed while the joints are to be broken. The disassembly precedence constraints are represented in a mathematical formation: two matrices and a list are used to represent the information on interferences between joints and components and among the components themselves.

- Joint accessibility matrix A . The rows correspond to joints and the columns to components. The element a_{ij} is binary and indicates whether Joint i is restricted by the presence of Component j .
- Disassembly precedence matrix P . The rows and columns correspond to components. The element p_{ij} indicates, for each Component i , which access directions are obstructed by Component j . If five possible access directions ($\pm x, \pm y, \pm z$) are considered, assuming that the product is fixed on a horizontal table or on the floor, each element p_{ij} can be expressed as five bits, each of which corresponds to one of the five access directions.
- Joints list for each component $L = (L_1, L_2, \dots, L_n)$. The element L_i indicates a list of all the joints that involve Component i .

The disassembly precedence constraints for the example in Fig. 3 are represented as follows:

$$A = \begin{Bmatrix} 0 & 0 & 1 \\ 0 & 0 & 0 \end{Bmatrix} \quad (4)$$

$$P = \begin{Bmatrix} 0 & 0010 & 0110 \\ 0001 & 0 & 0110 \\ 1001 & 1001 & 0 \end{Bmatrix} \quad (5)$$

FSG implemented in Problem 1

```

1 Set the vector of components available for disassembly  $C_A$  to  $\emptyset$ ;
2 For  $i = 0$  to length of  $Solution$ 
3   For  $r = 0$  to number of rules
4     If  $Fol_r$  has not been inserted to  $C_A$  and  $Pre_r \subseteq Solution$  then
5       Insert  $Fol_r$  into  $C_A$ ;
6     End if
7   End for
8   Randomly choose a component from  $C_A$ ;
9   Push the chose component back to  $Solution$ ;
10  Delete the chose component from  $C_A$ ;
11 End for
```

Fig. 4. The pseudo-code of FSG implemented in Problem 1.

$$L = \{L_1, L_2, L_3\} = \{\{j_1, j_2\}, \{j_1\}, \{j_2\}\} \quad (6)$$

The pseudo-code of the FSG implemented in Problem 2 is outlined in Fig. 5.

2.4. Teaching phase operator

TPO is used to replace current learners with new better learners in terms of better results (fitness function values) in the teaching phase. The teacher tries to improve the mean of the results of learners by giving his/her experiences and knowledge (e.g. better disassembly sequence) for the learners with expectation of their fitness function value to reach his/her level eventually. In other words, the feasible solutions in the teaching phase follow a random process to evolve and the quality of solutions depends on teacher's and population's capability.

New solutions can be randomly generated using TPO according to the teacher and the class in the teaching phase. Firstly the low bound of the objective function value (f_{lb}) is given according to the best disassembly situation. Then, a self-adaptive parameter named teaching factor (p_t) is used for modifying the existing learners, which can be calculated according to the mean of the objective function value of the population (f_m), the objective function value of the teacher (f_t) and the low bound of the objective function value:

$$g(x) = \frac{x}{x - f_{lb}} \quad (7)$$

$$p_t = \frac{g(f_t)}{g(f_t) + g(f_m)} \quad (8)$$

FSG implemented in Problem 2

```

1 Set the vector of components available for disassembly  $C_A$  to  $\emptyset$ ;
2 Set the vector of inaccessible joints  $J_I$  to {all the joints}
3 For  $i = 0$  to length of  $Solution$ 
4   For each component  $c$  in the product
5     If  $c \in C_A$  or  $c \in Solution$  then
6       continue;
7     End if
8     If  $c$  can be extracted by checking  $P$  then
9       push  $c$  back to  $C_A$ ;
10    End if
11  End for
12  For each joint  $j$  in the product
13    If  $j \notin J_I$  then
14      continue;
15    End if
16    If  $j$  is accessible by checking  $A$  then
17      erase  $j$  from  $J_I$ ;
18    End if
19  End for
20  For each component  $c$  in  $C_A$ 
21    If  $L_c \cap J_I \neq \emptyset$  then
22      erase  $c$  from  $C_A$ ;
23    End if
24  End for
25  Randomly Choose a component  $c$  from  $C_A$ ;
26  Push the  $c$  back to  $Solution$ ;
27  Delete  $c$  from  $C_A$ ;
28  Set all the elements in row  $c$  or column  $c$  in  $A$  and  $P$  to 0;
29 End for
```

Fig. 5. The pseudo-code of FSG implemented in Problem 2.

Finally, the precedence preservative operator is applied to preserve the precedence relationship in the feasible solutions when updating them. The pseudo-code of the TPO is outlined in Fig. 6.

In the procedure of TPO, a new solution is generated by a procedure of choosing and setting the elements one by one, from the left to the right of the solution. Firstly, the probability is set randomly, which is used to compare with p_t . If the probability is less than p_t , the leftmost element of the teacher is chosen and pushed back to the new solution; otherwise the leftmost element of the learner is chosen and pushed back to the new solution. Then, the chosen element is erased from both the teacher and learner. Finally, the learner is updated with the new generated solution if the new one is better. In this way, the disassembly precedence in solutions is preserved.

Table 1 gives an example for using TPO to generate a new solution with 5 elements: a new solution (2 3 4 5 1) is generated from a teacher (2 3 1 4 5) and a learner (4 3 5 1 2). Suppose Component 3 has precedence over Component 5 and $p_t = 0.6$. It is seen that the precedence is preserved in the new solution.

2.5. Learning phase operator

LPO is used to modify the existing learners with new better learners in the learning phase, where learners learn from each other and improve their abilities. Learning phase also follows a random process to generate feasible solutions, but the qualities of the solutions depend on current learner's ability, other learners' ability and random distribution.

New solutions can be randomly generated using LPO according to the learners themselves, other learners and randomly generated learners in the learner phase. Two self-adaptive parameters named self-learning factor (p_s) and factor of learning from another (p_a) are used for modifying the existing learners, which can be calculated according to the objective function value of the learner itself (f_s), the objective function value of another learner (f_a), the objective function value of a randomly generated learner (f_r) and the low bound of the objective function value (f_{lb}):

$$p_s = \frac{g(f_s)}{g(f_s) + g(f_a) + g(f_r)} \quad (9)$$

$$p_a = \frac{g(f_s) + g(f_a)}{g(f_s) + g(f_a) + g(f_r)} \quad (10)$$

TPO

```

1  For each element in a new Solution
2      Set probability ∈ [0, 1] randomly;
3      If probability <  $p_t$  then
4          Push Teacher's leftmost element back to Solution;
5      Else
6          Push Learner's leftmost element back to Solution;
7      End if
8      erase the chosen element from Teacher;
9      erase the chosen element from Learner;
10 End for
11 Calculate the objective function value of the new Solution;
12 If the new Solution is better than the Learner;
13     Update the Learner with the new Solution;
14 End if
```

Fig. 6. The pseudo-code of TPO.

Table 1

An example for using TPO to generate a new solution from a teacher and a learner with 5 elements.

Step	p_t	Probability	Teacher	Learner	Solution
1	0.6	0.5	(<u>2</u> 3 1 4 5)	(4 3 5 1 2)	(2)
2	0.6	0.2	(<u>3</u> 1 4 5)	(4 3 5 1)	(2 3)
3	0.6	0.8	(1 4 5)	(<u>4</u> 5 1)	(2 3 4)
4	0.6	0.7	(1 5)	(<u>5</u> 1)	(2 3 4 5)
5	0.6	0.3	(1)	(1)	(2 3 4 5 1)

Remarks: the underlined figure indicates the chose element in each step.

LPO

```

1  For each element in a new Solution
2      Set probability ∈ [0, 1] randomly;
3      If probability <  $p_s$  then
4          Push Teacher's leftmost element back to Solution;
5      Else if probability <  $p_a$  then
6          Push Another's leftmost element back to Solution;
7      Else
8          Push Generated's leftmost element back to Solution;
9      End if
10     erase the chosen element from Learner;
11     erase the chosen element from Another;
12     erase the chosen element from Generated;
13 End for
14 Calculate the objective function value of the new Solution;
15 If the new Solution is better than the Learner then
16     Update the Learner with the new Solution;
17 End if
```

Fig. 7. The pseudo-code of LPO.

The pseudo-code of LPO is similar to that of TPO outlined in Fig. 7. A new solution is generated from three parent solutions with two self-adaptive parameters in LPO, while a new solution is generated from two parent solutions with one self-adaptive parameter in TPO. In the learner phase, the learning factors for different learners are different, and they need to be calculated before implementing LPO, while the teaching factor just needs to be calculated once in the teaching phase.

2.6. Comparison of STLBO algorithm with other optimization algorithms

Like GA, PSO, TLBO, etc., the STLBO algorithm is also a population based technique which implements a group of solutions to evolve and learns to find a near optimum or optimum solution.

Compared with GA and PSO applied in solving DSP problems in the previous research works, the teaching-learning-based evolutionary mechanism of the STLBO algorithm is novel and different from the crossover and mutation mechanisms of GA and particle position update mechanism of PSO. In the STLBO algorithm, all the algorithm parameters (except population size and iteration times) are self-adapted and need not to be tuned. While GA requires the crossover rate and mutation probability, and the discrete PSO requires inertia weight and two acceleration constants.

The STLBO algorithm is also different from the TLBO algorithm in the following aspects. (a) The realization methods for evolutionary mechanism are different. In the STLBO algorithm, learners are modified using the TPO and LPO, applying the method of precedence preservation crossover operation. While in the TLBO

algorithm, learners are modified using multidimensional matrix computation. (b) The adaptation methods for the algorithm parameters are different. In the STLBO algorithm, the teaching factor (p_t), self-learning factor (p_s) and factor of learning from another (p_a) are calculated according to the results of learners (objective function values of solutions). While in the TLBO algorithm, the difference mean is calculated according to the grades of learners in each learning subject, in which the learning factor (r_t) is a random number in the range $[0, 1]$ and the teaching factor ($round(1 + r_t)$) can be either 1 or 2 decided by r_t .

2.7. Implementation of STLBO algorithm for DSP problems

The steps for implementation of the STLBO algorithm for solving DSP problems can be summarized as follows:

- Step 1: Define and formulate a DSP problem in a mathematical model and represent the disassembly precedence constraints;
- Step 2: Initialize the optimization parameters: solution length (number of components) population size (number of learners) and iteration times;
- Step 3: Generate a random population using FSG according to the population size and the number of components for disassembly;
- Step 4: Calculate the teaching factor and modify all the learners using TPO;
- Step 5: Calculate the self-learning factor and factor of learning from another for each learner and modify all of them using LPO;
- Step 6: Stop if the stop criteria are satisfied or maximum iteration times are achieved, otherwise repeat from Step 4.

3. Experimental studies

In order to test the performance of the STLBO algorithm and conduct a further comparative study, two kinds of disassembly sequence planning problems are described and used for testing. In both problems, the case of complete disassembly of wasted products is considered and it assumes that the structure of waste products and disassembly precedence constraints are known. Two types of representation methods of disassembly precedence constraints and the corresponding methods for generating a feasible disassembly sequence are in advance presented respectively in Section 2.3. All the algorithms tested were coded in the C++ programming language and carried out on a personal computer with 2 GHz Intel Core2 Duo CPU T5750 and 2 GB memory.

3.1. Problem 1

In the first DSP problem, all the components and joints in a waste product are uniformly considered as components. Four types of attributes of the disassembly process are considered: disassembly direction, disassembly method, demand after disassembly, and material contained in component [5,15]. A brief description of these attributes is presented as follows, where $x \in \{0, 1, \dots, n-1\}$ denotes the identifier of a component and n denotes the number of components.

- Disassembly direction: $dir(x) \in \{\pm x, \pm y, \pm z\}$. Each component must be disassembled in a particular direction. The $\pm x$, $\pm y$ and $\pm z$ indicate six possible directions respectively.
- Disassembly method: $met(x) \in \{D, N\}$. The letter D indicates Destructive disassembly, which focuses on recycling of materials, while the letter N indicates nondestructive disassembly, which focuses on components reuse.

- Demand after disassembling: $dem(x) \in \{0, 1, 2\}$. The values 0, 1 and 2 indicate no demand, demand of reuse and demand of recycling respectively.
- Material contained in a component: $mat(x) \in \{A, P, S\}$. The letters A, P and S indicate three types of materials contained in components: aluminum, plastic, and steel, respectively.

The objective of this problem is to find a feasible disassembly sequence $X = (x_0, x_1, \dots, x_{n-1})$ to minimize the total disassembly time $f(X)$, which depends on the basic disassembly time $bt(x_i)$, the penalty of direction change $dt(x_i)$ and the penalty of method change $mt(x_i)$ for each component x_i . The objective function is as follows:

$$\text{minimize} : f(X) = \sum_{i=0}^{n-1} bt(x_i) + \sum_{i=0}^{n-2} dt(x_i) + \sum_{i=0}^{n-2} mt(x_i) \quad (11)$$

where $bt(x_i)$, $dt(x_i)$ and $mt(x_i)$ are computed as follows:

If

$$dem(x_i) = dem(x_{i+1}) = 2 \text{ and } mat(x_i) = mat(x_{i+1}) \quad (12)$$

then

$$bt(x_i) = 0, dt(x_i) = 0, mt(x_i) = 0 \quad (13)$$

otherwise

$$bt(x_i) = bas(x_i) \quad (14)$$

$$dt(x_i) = \begin{cases} 0, & \text{direction is not changed} \\ 1, & \text{direction is changed by } 90^\circ \\ 2, & \text{direction is changed by } 180^\circ \end{cases} \quad (15)$$

$$mt(x_i) = \begin{cases} 0, & \text{method is not changed} \\ 1, & \text{method is changed} \end{cases} \quad (16)$$

Eq. (12) represents a special situation: two adjacent components in a disassembly sequence are made by the same material, and both are demanded for recycling. These two components will not be separated from each other and will be disassembled as a combination. In Eq. (14), $bas(x_i)$ indicates the basic disassembly time for Component x_i .

An example for Problem 1 is shown in Fig. 8, where the waste product is modeled by a tree and the disassembly precedence constraints are defined among the nodes. In this tree, the dashed-line arrow indicates that the component/subassembly at the start point has priority for disassembly over the component/subassembly at the end point. The tree is generated based on the Bill of Materials (BOM) and the disassembly precedence constraints can be acquired from the geometric information and connector information using a connector-knowledge-based approach [33]. The waste product has 13 components indexed by numbers 0, 1, ..., 12. The input data of the example is given in Table 2. And the disassembly precedence rules generated from Fig. 8 are shown as follows:

$$\text{Rule}_1: Pre_1 = \emptyset, Fol_1 = \{1, 2\};$$

$$\text{Rule}_2: Pre_2 = \{1, 2\}, Fol_2 = \{0, 3, 6, 8, 11\};$$

$$\text{Rule}_3: Pre_3 = \{3\}, Fol_3 = \{4, 5, 7\};$$

$$\text{Rule}_4: Pre_4 = \{5\}, Fol_4 = \{9, 10, 12\}.$$

3.1.1. Role test of TPO and LPO

To demonstrate the effectiveness of the proposed TPO or LPO, the following three cases are implemented:

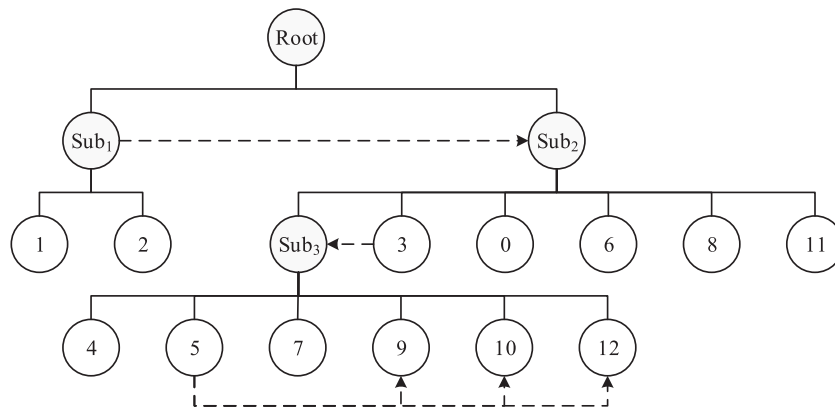


Fig. 8. Example for Problem 1 with tree structure and disassembly precedence constraints.

Table 2
Input data of the example for Problem 1.

x	0	1	2	3	4	5	6	7	8	9	10	11	12
bas(x)	3	4	1	1	2	2	2	1	2	1	2	1	3
dir(x)	Z	-X	X	X	-Y	Z	-Y	Z	-Y	X	-Y	-X	-Z
met(x)	D	D	D	D	N	N	N	N	D	N	D	N	D
dem(x)	1	1	1	0	0	1	2	1	2	0	1	2	1
mat(x)	S	S	P	S	P	P	S	A	A	A	S	A	S

Case 1: STLBO with TPO while without LPO;
Case 2: STLBO with LPO while without TPO;
Case 3: STLBO with both TPO and LPO.

The following parameters are initialized as:

- Solution length = 13;
- Population size = 10;
- Iteration times = 100.

The numerical results are shown in Fig. 9.

As shown in Fig. 9 (a), the algorithm in Case 1 converges fast to the local optimum at its early stage, and the objective function values $f(X)$ obtained in Case 1 are located in a large range in the first half of the iterations. Then all the learners converge fast to a certain level for that the teacher's ability and population's capability are limited.

As shown in Fig. 9 (b), the algorithm in Case 2 converges slow with low optimization accuracy, and the objective function values $f(X)$ obtained in Case 2 are located in a range during almost the whole iterations. When modifying the learners using LPO, a new solution is generated according to two existed learners and one randomly generated solution. As thus, there is a chance to find better solutions in Case 2, thus making the speed of convergence of the algorithm slow.

As shown in Fig. 9 (c), the complete the STLBO algorithm in Case 3 converges faster than the algorithms in Case 1 and Case 2 with higher optimization accuracy. The complete STLBO takes the advantages of both TPO and LPO.

From the numerical results, it can be concluded that learning from teacher using TPO makes the speed of convergence fast, and self-learning and learning from each other using LPO make additional chances to find better solutions. Combining TPO and LPO together makes the STLBO algorithm converges fast and enhances the search accuracy of it.

3.1.2. Comparison of STLBO algorithm with GA and SSO

The following cases were implemented to test and compare the STLBO algorithm with GA [5] and SSO algorithm [15]:

- Case 4: population size = 10 and iteration times = 100;
- Case 5: population size = 20 and iteration times = 50;
- Case 6: population size = 50 and iteration times = 20;
- Case 7: population size = 100 and iteration times = 10;
- Case 8: population size = 100 and iteration times = 100.

Firstly, Case 4 is used to compare the speed of convergence of the STLBO algorithm with that of GA and SSO algorithm. The optimum result of each iteration is recorded when running the algorithms. The convergence curves of these algorithms tested in Case 4 are shown in Fig. 10. It is observed that the STLBO algorithm converge faster than GA and SSO algorithm.

Then, Cases 4–8 are designed to compare the performances of the STLBO algorithm with the GA and SSO algorithm, and each algorithm runs 100 times independently in each case respectively. The numerical results are shown in Table 3, in which STD stands for Standard Deviation, ROB stands for Rate Of Best and ART stands for Average Running Time (millisecond). It is observed that the STLBO algorithm performs better than GA and SSO algorithm on all the cases, which means that the STLBO algorithm's teaching-learning-based mechanism with TPO and LPO is effective. The STLBO algorithm consumes a bit more time than SSO algorithm but it has a higher rate of finding the best solution.

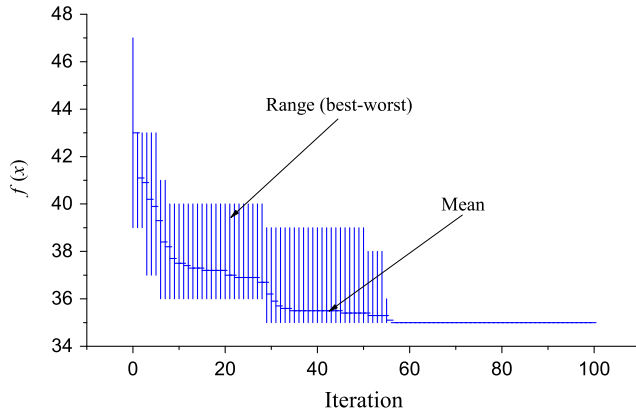
In addition, Cases 4–7 shows that a bigger population size makes STLBO, GA and SSO performing better even with less iteration times. As the value of iteration times in Case 7 is quite small, the performance of the algorithms may be improved by increasing the iteration times. Therefore Case 8 is implemented. It is observed that all these three algorithms' performance is improved by increasing the iteration times to 100.

One of the optimal solutions found by the STLBO algorithm for the example of Problem 1 is (1 2 3 0 8 11 5 7 9 6 4 10 12) and the total disassembly time is 33.

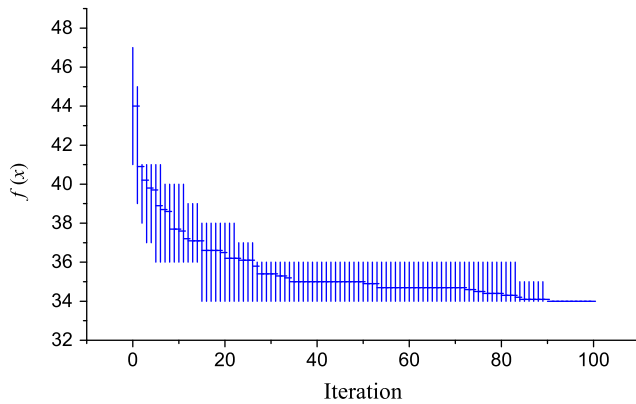
3.2. Problem 2

In the second DSP problem, the components and joints in a waste product are considered respectively. Three types of attributes of the disassembly process are considered as follows:

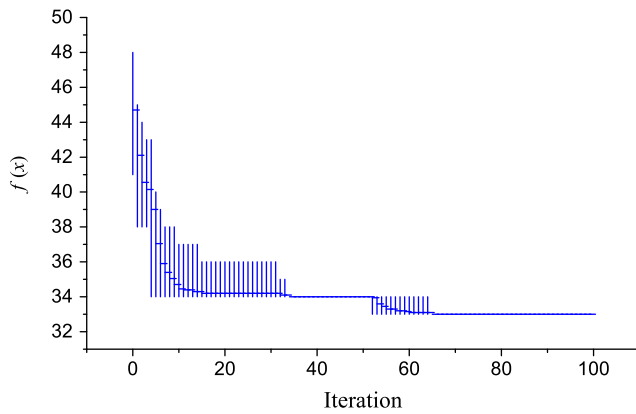
- (1) Direction changes of breaking joints. More direction changes lead to the higher cost of breaking joints;
- (2) Tool changes of breaking joints. More tool changes lead to the higher cost of breaking joints;
- (3) The number of directions obstructed by other components when extracting a component. More obstructed directions lead to the higher cost of extracting process.



(a) Case 1



(b) Case 2



(c) Case 3

Fig. 9. Range (best-worst-mean) of the obtained objective function values in Cases 1, 2 and 3.

The mathematical model of this DSP problem is presented in the benchmarks published by González and Adenso-Díaz [34]. The objective function is as follows:

$$\text{minimize : } f(X) = \sum_{i=0}^{n-1} (c_j(x_i) + c_c(x_i)) \quad (17)$$

where $c_j(x_i)$ indicates the cost of breaking all the joints that involve component x_i after $x_0, x_1, \dots, x_{n-1}$ are all extracted, and $c_c(x_i)$ indicates the cost of extracting component x_i after $x_0, x_1, \dots, x_{n-1}$ are all extracted. They are computed as follows:

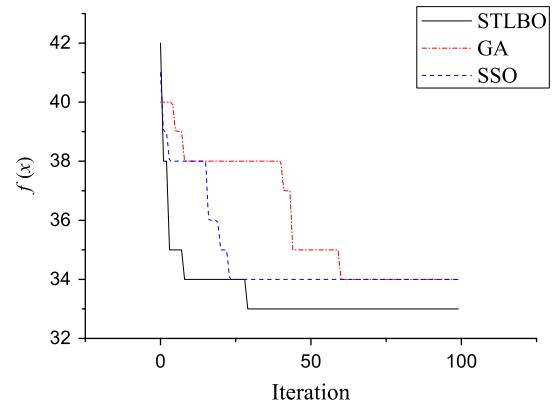


Fig. 10. Convergence curves of STLBO, GA and SSO in Case 4.

Table 3
Numerical results for Cases 4–8.

Algorithm	Case	Best	Worst	Mean	STD	ROB (%)	ART
STLBO	Case 4	33	36	33.72	0.60	35	1127
	Case 5	33	35	33.59	0.65	50	1201
	Case 6	33	35	33.51	0.52	50	1098
	Case 7	33	35	33.53	0.54	49	1078
	Case 8	33	34	33.14	0.35	86	11,732
GA	Case 4	33	38	35.48	1.02	2	184
	Case 5	34	38	35.18	0.95	0	198
	Case 6	33	37	34.51	0.94	8	291
	Case 7	33	36	34.23	0.68	8	569
	Case 8	33	35	33.80	0.61	31	5971
SSO	Case 4	33	36	33.75	0.56	30	966
	Case 5	33	36	33.77	0.60	31	993
	Case 6	33	36	33.78	0.59	30	964
	Case 7	33	36	33.73	0.63	36	965
	Case 8	33	34	33.16	0.37	83	9732

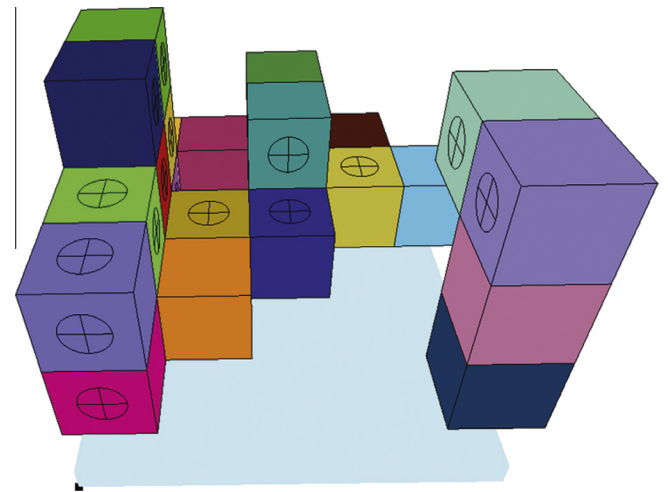


Fig. 11. Example of an instances (100-025-005-05) with 25 components in the published benchmarks.

$$c_j(x_i) = \sum_{j \in B_j(x_i)} c(j) \cdot c_1(j) \cdot c_2(j) \quad (18)$$

where $B_j(x_i)$ indicates the list of joints that involve component x_i after $x_0, x_1, \dots, x_{n-1}$ are all extracted, $c(j)$ indicates the based cost of breaking Joint j , $c_1(j)$ indicates the cost penalty of tool change when breaking Joint j , and $c_2(j)$ indicates the cost penalty of direction change when breaking Joint j .

Table 4

Objective function values of the previous best solutions in the published benchmarks, and of the solutions found by the STLBO algorithm.

No.	Instance	Number of components	Previous best cost	Best cost found by STLBO
1	100-025-005-05	25	2649.0699	2649.2542
2	101-025-009-20	25	2229.7465	<u>2229.7465</u>
3	102-025-012-40	25	2670.7680	2642.0680*
4	200-025-025-40	25	2503.6491	2486.1487*
5	201-025-037-40	25	2536.6414	2519.2001*
6	202-025-050-40	25	2350.0500	2343.5000*
7	800-025-005-40	25	3144.3953	3210.2789
8	301-025-009-20	25	2411.9750	<u>2411.9750</u>
9	302-025-012-05	25	2381.0032	<u>2381.0032</u>
10	400-025-025-05	25	2245.2589	<u>2245.2589</u>
11	401-025-037-05	25	2347.4863	2351.2366
12	402-025-050-05	25	2199.3607	<u>2199.3607</u>
13	103-050-010-05	50	5505.0209	5492.0913*
14	104-050-017-20	50	6069.4630	6141.9410
15	105-050-025-40	50	5793.1697	6018.2916
16	203-050-050-40	50	5374.3406	5456.9973
17	204-050-075-40	50	5592.0996	5489.2619*
18	905-050-100-40	50	5621.3563	5618.9372*
19	303-050-010-40	50	6178.4352	6324.8494
20	304-050-017-20	50	5422.0031	5339.4085*
21	305-050-025-05	50	4720.4362	4725.2437
22	403-050-050-05	50	4613.7832	4668.9782
23	404-050-075-05	50	4523.0915	4539.8000
24	405-050-100-05	50	4759.9078	4786.0266

Remarks: the value of cost with * in the right indicates it's better than the previous best one, and the underlined value of cost represents it's equal to the previous best one.

$$c_c(x_i) = 100 - (4 - n_o(x_i)) \times 24.75 \quad (19)$$

where $n_o(x_i)$ indicates the number of directions obstructed by other components when extracting component x_i after $x_0, x_1, \dots, x_{n-1}$ are all extracted. $c_1(j)$ and $c_2(j)$ are computed as follows:

$$c_1(j) = \begin{cases} 1.1, \text{broke a joint just before and} \\ \quad \text{the tool is changed} \\ 1.1, \text{extracted a component just before} \\ 1, \text{otherwise} \end{cases} \quad (20)$$

$$c_2(j) = \begin{cases} 1.15, \text{broke a joint just before and} \\ \quad \text{the direction is changed} \\ 1.15, \text{extracted a component just before} \\ 1, \text{otherwise} \end{cases} \quad (21)$$

The published disassembly benchmarks of 24 instances were taken for experimental studies. The best solutions and corresponding objective function values for these instances are kept

up-to-date online (http://coruxa.epsig.uniovi.es/~adenso/file_d.html). The test problems (of size $n = 25$ and 50) were generated using the GEN-PRODIS tool [34], which is able to generate an unlimited number of random test cases with specified characteristics (number of components and degree of compactness). All the components are right-angled parallelepiped, and a component may not be involved in more than eight joints and the total number of disassembly tools was set to five. Fig. 11 gives an example of an instances (100-025-005-05) with $n = 25$ components in the published benchmarks.

To test the benchmarks, the following parameters are initialized as:

- Solution length = n ;
- Population size = 20;
- Iteration times = 200.

The best results obtained by the STLBO algorithm compared with the previous best results are shown in Table 4. It is seen that the STLBO algorithm found eight new best solutions and five solutions that are equal to the previous best ones. Table 5 outlines the eight new best solutions found by the STLBO algorithm.

In summary, the experimental results illustrate that the developed STLBO algorithm performs well on DSP since it outperforms other methods such as GA and SSO and find 8 new best solutions out of 24 instances in the published benchmarks. The success of the STLBO algorithm could come from a better balance of global exploration and local exploitation, resulting from a powerful teaching–learning–based mechanism. The teaching phase is more like a kind of global exploration and learner phase is more like a kind of local exploitation. Moreover, all the parameters in the STLBO algorithm are self-adapted and need not to be tuned thus making the implementation simpler. And the teacher of each iteration is used to modify the learners thereby increasing the convergence rate.

4. Conclusions

This paper proposes a novel STLBO algorithm for solving DSP problems. The STLBO algorithm is divided into two phase: teaching phase and learning phase. And three new key operators are presented: FSG, TPO and LPO. The developed STLBO algorithm is a discrete and population-based optimization algorithm with a new teaching–learning–based evolutionary mechanism. The characteristics and advantages of the developed STLBO are summarized as follows:

- The STLBO algorithm is specially designed for DSP problems and adapts to different DSP problems with different ways of representing the disassembly precedence constraints.

Table 5

New best solutions found by the STLBO algorithm.

Instance	Best solution
102-025-012-40	(10 15 2 20 19 13 24 16 23 5 1 3 6 8 11 4 18 22 17 9 21 0 7 12 14)
200-025-025-40	(23 21 0 24 15 22 12 20 13 5 6 17 10 9 16 14 1 3 18 8 19 4 7 11 2)
201-025-037-40	(6 20 23 17 21 8 7 13 24 1 15 5 10 11 9 19 22 3 18 16 14 12 0 2 4)
202-025-050-40	(16 11 17 23 13 1 18 22 2 24 14 12 8 20 19 15 21 7 10 5 4 3 9 6 0)
103-050-010-05	(47 21 18 45 30 20 15 48 7 49 39 27 32 44 46 42 31 35 23 16 33 38)
204-050-075-40	37 36 41 2 22 28 26 43 9 12 19 25 40 17 10 24 13 29 3 14 8 6 11 5 34 0 1 4) (46 41 38 43 49 31 45 21 6 23 48 40 9 44 12 34 22 36 47 30 17 39)
905-050-100-40	33 28 2 5 42 37 15 29 25 10 7 35 26 3 20 19 32 14 11 8 13 0 24 27 16 1 4 18) (39 36 43 33 10 14 44 30 46 47 38 21 37 23 45 49 26 22 40 4 29 41)
304-050-017-20	42 24 19 12 32 35 18 48 3 20 6 11 28 15 27 34 5 31 16 2 17 0 7 1 8 25 9 13) (4 32 41 45 33 26 40 34 30 46 28 8 19 39 1 36 20 16 44 23 0 13 2 11) 17 22 49 25 47 18 37 48 12 27 42 21 38 10 14 3 5 7 43 35 9 6 31 15 24 29)

- The implementation of the STLBO algorithm is simple for that all the algorithm parameters are self-adapted and need not to be tuned. Thus the performance of the algorithm is stable.
- And STLBO can converge fast with high optimization accuracy by implementing the new teaching–learning-based evolutionary mechanism.

Two types of DSP problems are described and formulated in a mathematical model. The effectiveness and good performance of STLBO for solving DSP problems are proved by experimental studies and benchmark test.

In the future, more complex case studies for DSP problems will be introduced. The performance of the STLBO algorithm can be enhanced by improving the design of TPO and LPO by combining them with other search techniques. The STLBO algorithm can also be used to solve complex combinatorial optimization problems. Moreover, the STLBO algorithm can be extended as a multi-objective optimization algorithm to solve multi-objective disassembly sequence planning problems.

Acknowledgements

This research is carried out as a part of GREENet project which was supported by a Marie Curie International Research Staff Exchange Scheme Fellowship within the 7th European Community Framework Programme under Grant Agreement No. 269122. The paper reflects only the author's views and that the Union is not liable for any use that may be made of the information contained therein. This research work is also supported by the Special Funds for the Scientific and Technological Cooperation with EU from the Ministry of Science and Technology of China under Grant No. 1208 and the Natural Science Foundation of China (NSFC) under Grant No. 51121002.

References

- [1] A. Gungor, S.M. Gupta, Issues in environmentally conscious manufacturing and product recovery: a survey, *Comput. Indus. Eng.* 36 (4) (1999) 811–853.
- [2] A.J. Lambert, Disassembly sequencing: a survey, *Int. J. Prod. Res.* 41 (16) (2003) 3721–3759.
- [3] S.S. Smith, W.-H. Chen, Rule-based recursive selective disassembly sequence planning for green design, *Adv. Eng. Inform.* 25 (1) (2011) 77–87.
- [4] T.C. Kuo, H.C. Zhang, S.H. Huang, Disassembly analysis for electromechanical products: a graph-based heuristic approach, *Int. J. Prod. Res.* 38 (5) (2000) 993–1007.
- [5] E. Kongar, S.M. Gupta, Disassembly sequencing using genetic algorithm, *Int. J. Adv. Manuf. Technol.* 30 (5–6) (2006) 497–506.
- [6] F. Giudice, G. Fargione, Disassembly planning of mechanical systems for service and recovery: a genetic algorithms based approach, *J. Intell. Manuf.* 18 (3) (2007) 313–329.
- [7] T. Go, D. Wahab, M.A. Rahman, R. Ramli, A. Hussain, Genetically optimised disassembly sequence for automotive component reuse, *Expert Syst. Appl.* 39 (5) (2012) 5409–5417.
- [8] W. Hui, X. Dong, D. Guanhong, A genetic algorithm for product disassembly sequence planning, *Neurocomputing* 71 (13) (2008) 2720–2726.
- [9] J. Li, L. Khoo, S. Tor, A novel representation scheme for disassembly sequence planning, *Int. J. Adv. Manuf. Technol.* 20 (8) (2002) 621–630.
- [10] K.-K. Seo, J.-H. Park, D.-S. Jang, Optimal disassembly sequence using genetic algorithms considering economic and environmental aspects, *Int. J. Adv. Manuf. Technol.* 18 (5) (2001) 371–380.
- [11] J.R. Li, L.P. Khoo, S.B. Tor, An object-oriented intelligent disassembly sequence planner for maintenance, *Comput. Indus.* 56 (7) (2005) 699–718.
- [12] Y.-J. Tseng, F.-Y. Yu, F.-Y. Huang, A green assembly sequence planning model with a closed-loop assembly and disassembly sequence planning using a particle swarm optimization method, *Int. J. Adv. Manuf. Technol.* 57 (9–12) (2011) 1183–1197.
- [13] Y. Wang, J. Liu, Chaotic particle swarm optimization for assembly sequence planning, *Robot. Comput.-Integr. Manuf.* 26 (2) (2010) 212–222.
- [14] H. Lv, C. Lu, An assembly sequence planning approach with a discrete particle swarm optimization algorithm, *Int. J. Adv. Manuf. Technol.* 50 (5–8) (2010) 761–770.
- [15] W.-C. Yeh, Optimization of the disassembly sequencing problem on the basis of self-adaptive simplified swarm optimization, *IEEE Trans. Syst. Man Cybernet. Part A: Syst. Humans* 42 (1) (2012) 250–261.
- [16] W.-C. Yeh, Simplified swarm optimization in disassembly sequencing problems with learning effects, *Comput. Oper. Res.* 39 (9) (2012) 2168–2177.
- [17] W. Li, K. Xia, L. Gao, K.-M. Chao, Selective disassembly planning for waste electrical and electronic equipment with case studies on liquid crystal displays, *Robot. Comput.-Integr. Manuf.* 29 (4) (2013) 248–260.
- [18] J. Wang, J. Liu, Y. Zhong, A novel ant colony algorithm for assembly sequence planning, *Int. J. Adv. Manuf. Technol.* 25 (11–12) (2005) 1137–1143.
- [19] S. Agrawal, M. Tiwari, A collaborative ant colony algorithm to stochastic mixed-model u-shaped disassembly line balancing and sequencing problem, *Int. J. Prod. Res.* 46 (6) (2008) 1405–1429.
- [20] S.M. McGovern, S.M. Gupta, Ant colony optimization for disassembly sequencing with multiple objectives, *Int. J. Adv. Manuf. Technol.* 30 (5–6) (2006) 481–496.
- [21] B. Adenso-Díaz, S. García-Carbajal, S. Lozano, An efficient grasp algorithm for disassembly sequence planning, *OR Spect.* 29 (3) (2007) 535–549.
- [22] B. Adenso-Díaz, S. García-Carbajal, S.M. Gupta, A path-relinking approach for a bi-criteria disassembly sequencing problem, *Comput. Oper. Res.* 35 (12) (2008) 3989–3997.
- [23] C. Andrés, S. Lozano, B. Adenso-Díaz, Disassembly sequence planning in a disassembly cell context, *Robot. Comput.-Integr. Manuf.* 23 (6) (2007) 690–695.
- [24] R. Rao, V. Savsani, D. Vakharia, Teaching–learning-based optimization: an optimization method for continuous non-linear large scale problems, *Inform. Sci.* 183 (1) (2012) 1–15.
- [25] R. Rao, V. Savsani, D. Vakharia, Teaching–learning-based optimization: a novel method for constrained mechanical design optimization problems, *Comput.-Aid. Des.* 43 (3) (2011) 303–315.
- [26] R. Rao, V. Patel, An elitist teaching–learning-based optimization algorithm for solving complex constrained optimization problems, *Int. J. Indus. Eng. Comput.* 3 (4) (2012) 535–560.
- [27] R. Rao, V. Patel, Comparative performance of an elitist teaching–learning-based optimization algorithm for solving unconstrained optimization problems, *Int. J. Indus. Eng. Comput.* 4 (1) (2013) 29–50.
- [28] P. Pawar, R.V. Rao, Parameter optimization of machining processes using teaching–learning-based optimization algorithm, *Int. J. Adv. Manuf. Technol.* 67 (5–8) (2013) 995–1006.
- [29] R. Venkata Rao, V. Kalyankar, Parameter optimization of modern machining processes using teaching–learning-based optimization algorithm, *Eng. Appl. Artif. Intell.* 26 (1) (2013) 524–531.
- [30] R. Venkata Rao, V. Kalyankar, Parameter optimization of machining processes using a new optimization algorithm, *Mater. Manuf. Process.* 27 (9) (2012) 978–985.
- [31] S. Satapathy, A. Naik, K. Parvathi, High dimensional real parameter optimization with teaching learning based optimization, *Int. J. Indus. Eng. Comput.* 3 (5) (2012) 807–816.
- [32] T. Niknam, F. Golestaneh, M.S. Sadeghi, Multiobjective teaching–learning-based optimization for dynamic economic emission dispatch, *Syst. J. IEEE* 6 (2) (2012) 341–352.
- [33] J. Li, Q. Wang, P. Huang, H. Shen, A novel connector-knowledge-based approach for disassembly precedence constraint generation, *Int. J. Adv. Manuf. Technol.* 49 (1–4) (2010) 293–304.
- [34] B. González, B. Adenso-Díaz, A scatter search approach to the optimum disassembly sequence problem, *Comput. Oper. Res.* 33 (6) (2006) 1776–1793.